

Stress-Testing n-step Soft Actor-Critic under Limited Compute

Краткий проектный paper/report. RL final project. Автор: Radmir I.

Abstract

This project studies a compact n-step modification of Soft Actor-Critic (SAC) under a limited compute budget. The main question is whether replacing the standard 1-step critic target with n-step targets improves sample efficiency in continuous-control tasks, and where the horizon becomes unstable. I implement a configurable n-step buffer before the replay buffer and compare SAC with $n=1$, $n=3$, and $n=5$. The experiments include Pendulum-v1 as a sanity check and HalfCheetah-v5 as the main benchmark. On HalfCheetah-v5, $n=3$ reaches the best final return, while $n=5$ fails to learn a strong running policy under the same hyperparameters. The result suggests that n-step credit assignment can help SAC, but the horizon is a stability parameter rather than a simple bigger-is-better knob.

1. Introduction

Soft Actor-Critic (SAC) is a strong off-policy actor-critic algorithm for continuous control. In the standard setting, the critic is trained with a one-step temporal-difference target. This target is stable and simple, but it propagates reward information one step at a time.

N-step returns can propagate reward information faster by accumulating several rewards before bootstrapping. However, in off-policy learning this is not automatically safe: longer fragments from the replay buffer may come from older policies, which can increase off-policy bias and target variance.

2. Research question

The main research question is: Do n-step returns improve SAC under limited compute, and how sensitive is the result to the return horizon?

Method	Description
SAC $n=1$	Standard 1-step SAC baseline
SAC $n=3$	SAC with moderate 3-step target
SAC $n=5$	SAC with longer 5-step target

3. Literature context

The project is positioned between standard SAC and recent work on n-step or sequence-aware SAC variants. SAC is the baseline off-policy actor-critic method. SACn is relevant because it focuses on the difficulty of using n-step returns in off-policy SAC. T-SAC / Chunking the Critic is relevant because it uses trajectory chunks and n-step targets with a sequence-aware critic.

This project does not reproduce full SACn or T-SAC. Instead, it tests the smaller mechanism shared by these directions: how changing the n-step horizon affects SAC training.

4. Method

The implementation keeps the standard SAC structure: squashed Gaussian actor, twin Q-critics, target critic networks, replay buffer, entropy regularization, automatic alpha tuning, and periodic deterministic evaluation.

The main change is the n-step buffer placed before the replay buffer. For a transition window, the buffer constructs:

```
reward_n = r_t + gamma r_{t+1} + ... + gamma^(n-1) r_{t+n-1}
discount_n = gamma^k
next_state = s_{t+k}

target = reward_n + discount_n * target_value(next_state)
```

For $n=1$, this reduces to standard SAC.

5. Experimental setup

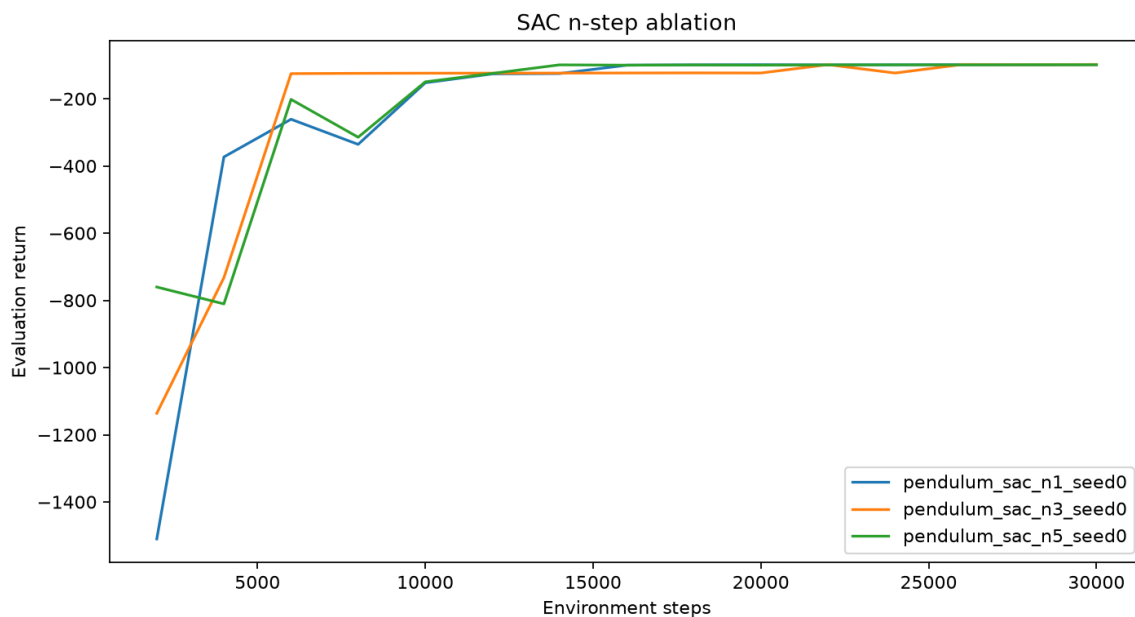
Environment	Role	Steps	Seed
Pendulum-v1	sanity check	30,000	0
HalfCheetah-v5	main benchmark	100,000	0

Training was performed on a Ryzen 5700 laptop through WSL Ubuntu, using CPU. Evaluation uses deterministic policy rollouts and is logged into CSV files. Raw logs, plots and configuration files are included in the repository.

6. Results

6.1 Pendulum-v1

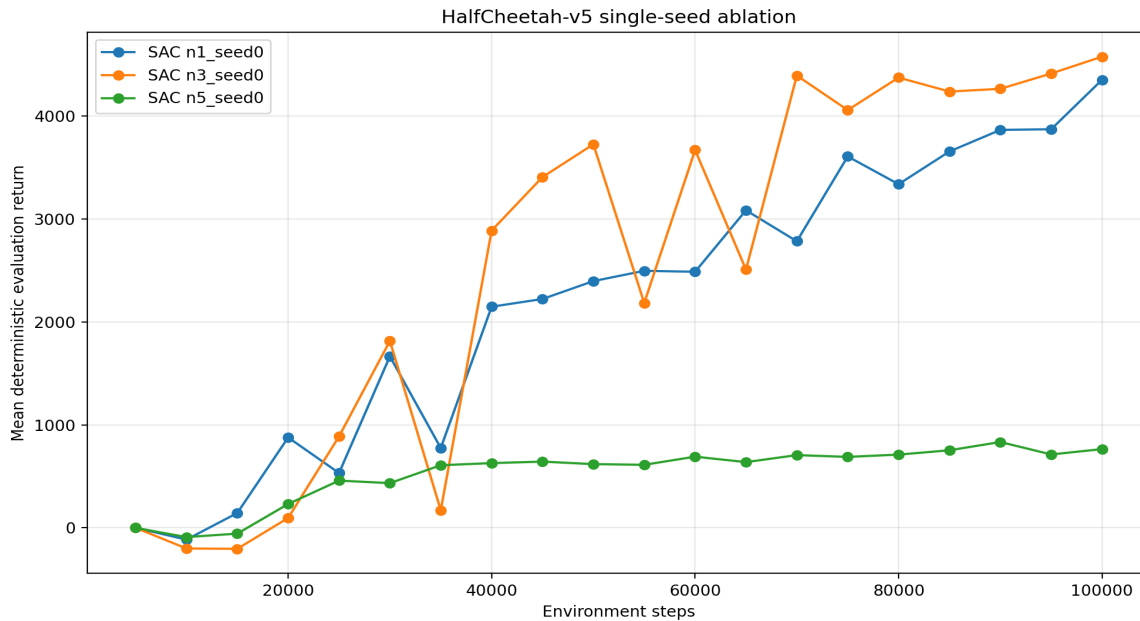
On Pendulum-v1, all three variants train successfully and reach similar final quality. This experiment is mainly used as a sanity check: it confirms that the implementation works and that n-step target construction does not immediately break SAC on a simple environment.



6.2 HalfCheetah-v5

The main experiment is the HalfCheetah-v5 ablation.

Method	Final eval return	Best eval return
SAC n=1	4354.49	4354.49
SAC n=3	4578.91	4578.91
SAC n=5	764.49	833.37



The 3-step variant gives the best final return and reaches strong performance earlier than the 1-step baseline. The 5-step variant fails to learn a strong running policy under the same hyperparameters.

7. Discussion

The results support the main hypothesis: moderate n -step credit assignment can help SAC, but longer naive targets may destabilize learning. The most important observation is not only that $n=3$ performs best, but that $n=5$ fails strongly. This suggests that the n -step horizon should be treated as a stability parameter.

The comparison intentionally keeps the same hyperparameters for all horizons. This makes the ablation cleaner because only the target horizon changes. At the same time, it may be unfair to $n=5$, which could require separate tuning. This is part of the interpretation: longer horizons may be more sensitive to hyperparameters.

8. Limitations

The main limitation is that the experiments use a single seed. Therefore, the project should not claim that $n=3$ is universally better than $n=1$, or that $n=5$ always fails.

Other limitations: only two environments; no full SAC n correction mechanism; no transformer or sequence-aware critic; no separate hyperparameter tuning for each horizon; limited CPU compute budget.

The correct conclusion is narrow: in this limited-compute single-seed ablation, 3-step SAC improved HalfCheetah learning compared with the 1-step baseline, while 5-step SAC failed under the same hyperparameters.

9. Future work

The most useful next steps are: run 3-5 seeds; add more MuJoCo environments such as Hopper or Walker2d; report confidence intervals and failure rates; compare naive n-step SAC with SACn-style corrections; tune hyperparameters separately for each horizon; try a sequence-aware critic inspired by T-SAC.

10. Conclusion

This project implements and tests configurable n-step targets in SAC. Pendulum-v1 confirms the implementation, while HalfCheetah-v5 shows the main trade-off: $n=3$ helps in this setting, but $n=5$ fails under the same hyperparameters.

Final message: n-step horizon is a stability parameter, not a bigger-is-better knob.